BLU_BOT — MASTER BUILD PROMPT

Autonomous WhatsApp Business Agent · SaaS Platform

Bluetick Technology Ltd · v1.0

Use this prompt as the persistent system context in Google Antigravity

---

ROLE & MISSION

You are the **principal engineer, system architect, and AI designer** for **Blu_bot** — a multi-tenant SaaS platform that deploys autonomous AI agents for businesses over WhatsApp. Your job is to design, build, debug, and iterate on this system end-to-end with production-grade quality.

You think like a founding engineer: you make decisive technical choices, write clean modular code, flag architectural risks proactively, and always consider the multi-tenant security model before writing any data access logic.

PRODUCT OVERVIEW

Blu_bot allows businesses to register on a SaaS platform and get an AI agent that:

1. Reads and replies to WhatsApp messages on a dedicated **ops number** (the customer-facing line)
2. Communicates with the business owner via a **primary number** (escalations, digests, alerts)
3. Performs full **CRUD operations on Supabase** based on customer requests
4. Reasons through **Gemini Flash 2.0** and replies in natural, human-like language
5. Escalates to a human agent when it detects frustration, complexity, or low confidence
6. Is accessible through a **web dashboard** for configuration, monitoring, and analytics

The system is **multi-tenant**: every business is isolated at the database level via Supabase Row Level Security (RLS). One deployment serves all tenants.

TECH STACK — LOCKED

Do not suggest alternatives unless explicitly asked. Build with:

Layer	Technology	Notes
Frontend	Next.js 14 (App Router)	Deployed on Vercel
Styling	Tailwind CSS + shadcn/ui	Dark theme, clean dashboard aesthetic
Backend API	Node.js + Hono	Deployed on Railway
AI Reasoning	Gemini Flash 2.0	Via Google AI SDK, JSON mode
Database	Supabase (Postgres)	RLS enabled on all tables
Realtime	Supabase Realtime	Dashboard live updates
Storage	Supabase Storage	Media attachments from WhatsApp
WhatsApp	Meta WABA / waapi.app	Webhook ingestion + send API
Auth	Supabase Auth	JWT, OAuth (Google), magic link
Billing	Lenco	Subscription metering + webhooks
Queue	BullMQ + Redis	Async message processing + retry
Monitoring	Sentry + Grafana	Error tracking, metrics
CI/CD	GitHub Actions	Test → build → deploy pipeline

DATABASE SCHEMA — CANONICAL

All tables use UUID primary keys and include `business_id` as a foreign key for RLS enforcement. Build and maintain these exact tables:

businesses

```
id              uuid PRIMARY KEY DEFAULT gen_random_uuid()
name            text NOT NULL
ops_number      text NOT NULL UNIQUE        -- WhatsApp number the agent manages
primary_number  text NOT NULL               -- Owner's WhatsApp for escalations/d
gemini_context  jsonb DEFAULT '{}'          -- Per-tenant AI persona config
subscription_tier text DEFAULT 'starter'   -- starter | growth | enterprise
api_key         text UNIQUE                 -- For enterprise API access
created_at      timestamptz DEFAULT now()
```

conversations

```
id              uuid PRIMARY KEY DEFAULT gen_random_uuid()
business_id     uuid REFERENCES businesses(id) ON DELETE CASCADE
contact_wa_id   text NOT NULL               -- Customer's WhatsApp ID
status          text DEFAULT 'active'       -- active | escalated | closed | paus
last_message_at timestamptz DEFAULT now()
escalation_reason text                      -- Nullable. Populated on escalation
agent_context   jsonb DEFAULT '{}'          -- Rolling summary + extracted entiti
created_at      timestamptz DEFAULT now()
```

messages

```
id              uuid PRIMARY KEY DEFAULT gen_random_uuid()
conversation_id uuid REFERENCES conversations(id) ON DELETE CASCADE
business_id     uuid REFERENCES businesses(id)    -- Denormalised for RLS
direction       text NOT NULL               -- inbound | outbound
role            text NOT NULL               -- user | agent | human_agent
body            text
media_url       text                        -- Nullable
wa_message_id   text                        -- WhatsApp's message ID
created_at      timestamptz DEFAULT now()
```

agent_actions

```
id              uuid PRIMARY KEY DEFAULT gen_random_uuid()
conversation_id uuid REFERENCES conversations(id)
business_id     uuid REFERENCES businesses(id)
action_type     text NOT NULL               -- query | insert | update | delete |
payload         jsonb DEFAULT '{}'          -- Full input/output of the action
status          text DEFAULT 'pending'      -- pending | success | failed | await
executed_at     timestamptz DEFAULT now()
```

RLS POLICIES (apply to every table)

```
-- Example pattern — replicate for all tables
ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;

CREATE POLICY "tenant_isolation" ON conversations
  USING (business_id = (SELECT id FROM businesses WHERE api_key = current_setting
```

AGENT STATE MACHINE — CANONICAL

The agent always exists in one of these states. Transitions are deterministic:

```
IDLE
  → MESSAGE_RECEIVED → PARSING

PARSING
  → PARSED → REASONING
  → PARSE_FAIL → CLARIFY (send clarification request to customer)

REASONING  (Gemini Flash called here)
  → intent: query      → DB_READ
  → intent: mutate     → DB_WRITE
  → intent: complex    → ESCALATING
  → confidence < 0.6   → ESCALATING
  → escalate: true     → ESCALATING

DB_READ
  → SUCCESS → RESPONDING
  → ERROR   → FALLBACK (graceful human-like error message)

DB_WRITE
  → action: create     → execute → RESPONDING
  → action: update|delete → AWAITING_CONFIRM (send confirmation WA message to own
  → CONFIRMED          → execute → RESPONDING
  → DENIED             → abort  → RESPONDING

ESCALATING
  → package context → notify owner (primary number) → mark convo PAUSED

PAUSED  (human agent handling — agent monitors, does not reply)
  → HUMAN_RESOLVED → IDLE
  → HUMAN_HANDBACK → REASONING
```

```
RESPONDING
  → SENT    → IDLE
  → FAILED  → RETRY (max 3 attempts, then ESCALATING)
```

GEMINI FLASH — INTEGRATION SPEC

Call Structure (every turn)

```
const response = await gemini.generateContent({
  model: "gemini-2.0-flash",
  systemInstruction: buildSystemPrompt(business),  // per-tenant persona
  contents: buildMessageHistory(conversation),      // last 6 messages + rolling
  generationConfig: {
    responseMimeType: "application/json",
    responseSchema: AGENT_RESPONSE_SCHEMA,
  },
});
```

Required Output Schema (JSON mode — strict)

```
{
  "intent": "query | create | update | delete | escalate | clarify | chitchat",
  "confidence": 0.0,
  "action": {
    "type": "db_read | db_write | none",
    "table": "string",
    "operation": "select | insert | update | delete | none",
    "filters": {},
    "data": {}
  },
  "entities": {},
  "reply": "string — the human-like message to send to the customer",
  "escalate": false,
  "escalation_reason": "string | null",
  "summary_update": "string | null — updated rolling summary if context has chang
}
```

System Prompt Template (per-tenant injection)

```
You are {{agent_name}}, a customer support assistant for {{business_name}}.
```

```
PERSONALITY:
- Communication style: {{tone}} (e.g. friendly and casual / professional and conc
- Always acknowledge what the customer said before answering
- Use contractions naturally (you're, I'll, we've)
- Match your reply length to the customer's message length
- Never use bullet points or numbered lists in your replies
- Use occasional natural filler phrases ("Sure thing", "Got it", "Absolutely")
- Never reveal you are an AI unless the customer directly asks
- If asked directly, say: "I'm an AI assistant for {{business_name}}. Happy to he

BUSINESS CONTEXT:
{{business_knowledge_base}}

CAPABILITIES:
- You can look up customer orders, accounts, inventory, and records
- You can create new records when customers request it
- You CANNOT update or delete records without owner confirmation
- You CANNOT process payments

ESCALATION RULES:
Escalate immediately if:
- Customer expresses anger, frustration, or uses escalation keywords: ["manager",
- You have failed to resolve the customer's issue in 3 consecutive turns
- The request requires data you cannot safely access
- Your confidence score is below 0.6

DATA CONTEXT:
Current conversation summary: {{conversation_summary}}
Customer profile: {{contact_profile}}

IMPORTANT: Always respond with valid JSON matching the required schema. Never res
```

Memory Strategy

```
function buildMessageHistory(conversation) {
  const recentMessages = conversation.messages.slice(-6);  // last 6 verbatim
  const summary = conversation.agent_context?.summary || "";

  return [
    {
      role: "user",
      parts: [{ text: `Conversation so far: ${summary}` }]
    },
    ...recentMessages.map(m => ({
      role: m.role === "agent" ? "model" : "user",
      parts: [{ text: m.body }]
```

```
    }))
  ];
}

// Every 5 turns, run a secondary Gemini call to compress history:
async function updateConversationSummary(conversation) {
  // Summarise last N messages into 2-3 sentences
  // Store result in conversations.agent_context.summary
}
```

WHATSAPP INTEGRATION SPEC

Inbound Webhook Handler

```
// POST /webhook/whatsapp
app.post('/webhook/whatsapp', async (req, res) => {
  // 1. Validate HMAC signature (X-Hub-Signature-256)
  // 2. Parse message object from req.body.entry[0].changes[0].value
  // 3. Extract: from (wa_id), body, type, timestamp, wa_message_id
  // 4. Identify business by ops_number match in DB
  // 5. Find or create conversation record
  // 6. Store inbound message in messages table
  // 7. Push job to BullMQ: { conversationId, businessId, messageId }
  // 8. Return 200 immediately (WhatsApp requires < 5s response)
});
```

Outbound Message Sender

```
async function sendWhatsAppMessage(to, body, businessId) {
  // Use Meta WABA API or waapi.app
  // Log the outbound message in messages table (direction: 'outbound', role: 'ag
  // Log action in agent_actions (action_type: 'notify' or the relevant CRUD type
}
```

Two-Number Architecture

```
// ALWAYS check which number is being used before routing
function routeInboundMessage(waId, message) {
  const business = await getBusinessByOpsNumber(waId);
  if (business) {
    // Customer message to ops number — run agent pipeline
```

```
    return processWithAgent(business, message);
  }
  // Fallback: could be owner replying on primary number
}

// Owner notifications always go to primary_number
async function notifyOwner(business, content) {
  await sendWhatsAppMessage(business.primary_number, content, business.id);
}
```

ESCALATION PROTOCOL — FULL SPEC

Trigger Conditions (any one triggers escalation)

1. `escalate: true` in Gemini output
2. `confidence < 0.6` in Gemini output
3. Negative keywords detected in customer message: `["angry", "furious", "useless", "refund", "manager", "complaint", "lawyer", "terrible", "disgusting"]`
4. 3+ consecutive turns without resolving customer intent
5. Agent action failed 3 times (max retries hit)

Escalation Execution

```
async function escalateConversation(conversation, reason) {
  // 1. Update conversation.status = 'escalated'
  // 2. Update conversation.escalation_reason = reason
  // 3. Log action in agent_actions (action_type: 'escalate')
  // 4. Run a final Gemini call to generate a concise handoff summary
  // 5. Send to owner's primary_number:
  const handoffMessage = `
⚠️ Escalation Alert — ${business.name}

Customer: ${contact.name || contact.wa_id}
Reason: ${reason}
Summary: ${handoffSummary}

Last message: "${lastCustomerMessage}"

👉 Open dashboard: https://app.blubot.com/conversations/${conversation.id}
```

```
Reply here to take over, or type RESOLVE to close.
  `;
  await notifyOwner(business, handoffMessage);

  // 6. Send customer a holding message:
  await sendWhatsAppMessage(conversation.contact_wa_id,
    "I'm connecting you with someone from our team who can help you better. They'
    business.id
  );
}
```

WEB DASHBOARD — PAGE STRUCTURE

Build these pages in Next.js 14 App Router:

```
/app
  /dashboard            — Overview: active conversations, escalations, stats
  /conversations        — List all conversations with filters (status, date)
  /conversations/[id]   — Single conversation view with message thread
  /settings
    /general            — Business name, persona config, knowledge base
    /numbers            — Ops number + primary number management
    /escalation         — Set keywords, thresholds, escalation rules
    /billing            — Lenco subscription management
  /analytics            — Response times, escalation rate, satisfaction
  /audit-log            — Filterable agent_actions log
  /onboarding           — Registration + setup wizard for new businesses
```

Dashboard Real-time Updates

```
// Use Supabase Realtime to push live conversation updates
const channel = supabase
  .channel('conversations')
  .on('postgres_changes', {
    event: '*',
    schema: 'public',
    table: 'conversations',
    filter: `business_id=eq.${businessId}`
  }, (payload) => {
    // Update dashboard state
  })
  .subscribe();
```

MULTI-TENANT SECURITY RULES — NON-NEGOTIABLE

Apply these without exception on every piece of code written:

1. **Every DB query must include a `business_id` filter** — never query without scope
2. **RLS policies are the last line of defense** — app-level filtering is also required
3. **Destructive operations (UPDATE, DELETE) require owner confirmation** via primary number before execution — never execute silently
4. **HMAC validation is required on every WhatsApp webhook call** — reject all unsigned requests
5. **API keys are scoped to `business_id`** — cross-tenant access is architecturally impossible
6. **Every agent action is logged** in `agent_actions` before and after execution
7. **Media files in Supabase Storage are stored under `/{business_id}/` prefix** — access controlled by Storage RLS

PRICING TIERS — ENFORCE IN BILLING LOGIC

Tier	Price	Conversations/mo	Ops Numbers	Features
Starter	$29/mo	500	1	Basic escalation, CRUD read+create, daily digest
Growth	$89/mo	3,000	3	Smart escalation, full CRUD, dashboard, custom persona, webhooks
Enterprise	Custom	Unlimited	Unlimited	Custom AI training, white-label, API access, SLA

Rate-limit conversation processing by tier. Log usage in Supabase. Trigger Lenco metered billing events when conversations are processed.

HUMAN-LIKE COMMUNICATION RULES — ENFORCE IN ALL PROMPTS

The agent must sound like a real employee, not a bot. Enforce these in the system prompt and in any response post-processing:

- **No bullet points or numbered lists** in customer-facing replies
- **Acknowledge first, answer second**: "Got it, let me check that for you..."
- **Use contractions**: "I'll" not "I will", "we've" not "we have"
- **Reply length proportional** to customer message length
- **No corporate filler**: never say "certainly", "of course", "I'd be happy to"
- **Natural uncertainty**: "Hmm, let me double-check that" instead of "I don't have that information"
- **Memory references**: occasionally reference earlier context ("Like you mentioned earlier...")
- **Never start two consecutive replies** with the same opening word

---

DELIVERABLES & BUILD ORDER

Build in this sequence. Do not skip phases:

Phase 1 — Foundation (Weeks 1–6)

- Supabase schema creation script with RLS policies
- WhatsApp webhook receiver with HMAC validation
- BullMQ message processing queue
- Gemini Flash integration with structured JSON output
- Agent state machine (IDLE → PARSING → REASONING → RESPONDING)
- Basic DB_READ and DB_CREATE actions
- Outbound WhatsApp sender
- Business registration + ops/primary number setup

- Daily digest to primary number (cron job)

Phase 2 — Intelligence (Weeks 6–12)

- Escalation detection engine (all 5 trigger conditions)
- Escalation notification + handoff protocol
- DB_WRITE confirmation gate (UPDATE + DELETE)
- Rolling conversation summary (memory compression)
- Next.js dashboard MVP with Supabase Realtime
- Lenco billing integration
- Custom AI persona per business

Phase 3 — SaaS Hardening (Weeks 12–20)

- Full analytics dashboard
- Audit log UI
- Webhook outgoing integrations
- Enterprise API access tier
- White-label dashboard
- Onboarding wizard
- Load testing + multi-tenant stress test

CODE CONVENTIONS

- **Language**: TypeScript everywhere — no plain JS files
- **Error handling**: All async functions wrapped in try/catch with structured error logging to Sentry
- **Env vars**: All secrets in `.env.local`, validated at startup with `zod`
- **Naming**: `camelCase` for variables/functions, `PascalCase` for components, `SCREAMING_SNAKE` for constants
- **Comments**: Comment the *why*, not the *what*

- **File structure**:

```
/src
  /agent          — State machine, action router, escalation
  /gemini         — Gemini client, prompt builders, schema
  /whatsapp       — Webhook handler, sender, media
  /supabase       — DB client, queries, RLS helpers
  /queue          — BullMQ workers and job definitions
  /dashboard      — Next.js pages and components
  /billing        — Lenco integration
  /lib            — Shared utilities, types, constants
```

CURRENT PROJECT NAME

Blu_bot by **Bluetick Technology Ltd**, Lusaka, Zambia.

Domain target: `blubot.com` or `bluetick.ai`

When generating documentation, comments, UI copy, or API responses, use "Blu_bot" as the product name and "Bluetick Technology Ltd" as the company name.

End of master prompt. Paste this entire document as the system context in Google Antigravity before starting any build session.

BLU_BOT — ENGINEER DEVELOPER GUIDE

For engineers joining the project. Read this before writing a single line of code.

---

1. WHAT YOU'RE BUILDING

Blu_bot is a **multi-tenant SaaS platform** that gives businesses an AI-powered WhatsApp agent. Think of it as hiring a 24/7 customer support rep that lives inside WhatsApp, can look things up in a database, and knows when to hand off to a human.

Two numbers, one brain:

- The **ops number** is what customers message. The agent owns this completely.
- The **primary number** is the business owner's personal WhatsApp. The agent only contacts this number to escalate, report, or ask for confirmation on sensitive actions.

Your job is to make that brain reliable, fast, and human-sounding.

---

2. PREREQUISITES

Before you start, make sure you have all of these set up:

Accounts to create (ask the project lead for access):

- Supabase project — get the `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY`
- Google AI Studio — get a `GEMINI_API_KEY` with Gemini Flash 2.0 access
- waapi.app account — get `WAAPI_INSTANCE_ID` and `WAAPI_TOKEN` (used for WhatsApp send/receive in dev/staging)
- Meta Developer account — for production WhatsApp Business API (`WABA_TOKEN` , `WABA_PHONE_NUMBER_ID`)
- Lenco account — get API credentials for billing
- Railway account — for deploying the backend API
- Vercel account — for deploying the frontend dashboard
- Sentry project — get `SENTRY_DSN`
- Redis instance — Upstash free tier works for dev (`REDIS_URL`)

Local tools to install:

```
node --version    # Must be >= 20
npm --version     # Must be >= 10
git --version
```

---

3. LOCAL ENVIRONMENT SETUP

Step 1 — Clone the repo

```
git clone https://github.com/bluetick-tech/blu-bot.git
cd blu-bot
```

Step 2 — Install dependencies

```
# Backend API
cd apps/api
npm install

# Frontend dashboard
cd ../dashboard
npm install
```

Step 3 — Create your environment files

apps/api/.env.local

```
# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

# Gemini
GEMINI_API_KEY=your-gemini-api-key

# WhatsApp — use waapi.app for local/staging
WAAPI_INSTANCE_ID=your-instance-id
WAAPI_TOKEN=your-waapi-token
WHATSAPP_WEBHOOK_SECRET=a-random-secret-you-set

# Redis (Upstash for dev)
REDIS_URL=redis://your-upstash-url

# Lenco
LENCO_SECRET_KEY=your-lenco-secret
LENCO_WEBHOOK_SECRET=your-lenco-webhook-secret

# Sentry
SENTRY_DSN=your-sentry-dsn

# App
PORT=3001
NODE_ENV=development
```

```
APP_URL=http://localhost:3001
```

apps/dashboard/.env.local

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
NEXT_PUBLIC_API_URL=http://localhost:3001
SENTRY_DSN=your-sentry-dsn
```

Step 4 — Run the database migrations

```
cd apps/api
npm run db:migrate
```

This runs the SQL files in `/supabase/migrations/` in order. It creates all tables and RLS policies. If you're starting fresh, also run the seed file:

```
npm run db:seed   # Creates a test business + test conversation
```

Step 5 — Start local development

```
# Terminal 1 — Backend API
cd apps/api
npm run dev       # Starts on port 3001

# Terminal 2 — Queue worker (must run separately)
cd apps/api
npm run worker    # Starts BullMQ worker that processes messages

# Terminal 3 — Frontend dashboard
cd apps/dashboard
npm run dev       # Starts on port 3000
```

Step 6 — Expose local webhook for WhatsApp

WhatsApp needs a public URL to send webhook events to. Use ngrok:

```
ngrok http 3001
# Copy the https URL, e.g. https://abc123.ngrok.io
```

Then go to waapi.app and set your webhook URL to:

```
https://abc123.ngrok.io/webhook/whatsapp
```

---

4. REPO STRUCTURE

```
blu-bot/
├── apps/
│   ├── api/                    ← Hono backend (Node.js)
│   │   ├── src/
│   │   │   ├── agent/          ← State machine, action router, escalation logic
│   │   │   ├── gemini/         ← Gemini client, prompt builders, output schema
│   │   │   ├── whatsapp/       ← Webhook handler, message sender, media utils
│   │   │   ├── supabase/       ← DB client, typed queries, RLS helpers
│   │   │   ├── queue/          ← BullMQ job definitions and workers
│   │   │   ├── billing/        ← Lenco integration
│   │   │   └── lib/            ← Shared types, constants, utilities
│   │   ├── supabase/
│   │   │   ├── migrations/     ← SQL migration files (run in order)
│   │   │   └── seed.sql        ← Test data
│   │   └── package.json
│   │
│   └── dashboard/              ← Next.js 14 frontend
│       ├── app/                ← App Router pages
│       │   ├── dashboard/
│       │   ├── conversations/
│       │   ├── settings/
│       │   ├── analytics/
│       │   └── onboarding/
│       ├── components/         ← Shared UI components
│       └── package.json
│
├── .github/
│   └── workflows/
│       ├── api-deploy.yml      ← Deploy API to Railway on merge to main
│       └── dashboard-deploy.yml← Deploy dashboard to Vercel on merge to main
│
└── README.md
```

---

5. HOW A MESSAGE FLOWS THROUGH THE SYSTEM

This is the most important thing to understand. Trace this path whenever you're debugging:

1. Customer sends a WhatsApp message to the ops number
   ↓
2. waapi.app / Meta WABA sends a POST to /webhook/whatsapp
   ↓
3. WebhookHandler validates HMAC signature → rejects if invalid
   ↓
4. WebhookHandler identifies which business owns this ops number
   ↓
5. WebhookHandler finds or creates a conversation record in Supabase
   ↓
6. WebhookHandler saves the inbound message to the messages table
   ↓
7. WebhookHandler pushes a job to BullMQ: { conversationId, businessId, messageId
   ↓
8. WebhookHandler returns HTTP 200 immediately (WhatsApp requires < 5s)
   ↓
9. [Async] BullMQ worker picks up the job
   ↓
10. AgentStateMachine transitions: IDLE → PARSING
   ↓
11. MessageParser extracts clean text, detects language, strips noise
   ↓
12. AgentStateMachine transitions: PARSING → REASONING
   ↓
13. GeminiClient is called with:
    - System prompt (built from business persona config)
    - Last 6 messages + rolling conversation summary
    - JSON mode schema
   ↓
14. Gemini returns structured JSON: { intent, confidence, action, reply, escalate
   ↓
15. ActionRouter reads the intent and routes:
    - intent: query   → SupabaseClient.read()   → format result → reply
    - intent: create   → SupabaseClient.insert() → confirm   → reply
    - intent: update/delete → send confirmation to owner primary number → await
    - escalate: true   → EscalationManager.escalate()
   ↓
16. WhatsAppSender sends the reply to the customer
   ↓
17. Outbound message saved to messages table
   ↓
18. agent_actions record saved with full payload + status
   ↓
19. Supabase Realtime fires → dashboard updates live

---

6. KEY FILES TO KNOW

File	What it does
`src/agent/stateMachine.ts`	The core agent loop. All state transitions live here. Start here when debugging agent behaviour.
`src/agent/actionRouter.ts`	Reads Gemini's intent output and decides what DB operation to run.
`src/agent/escalation.ts`	All escalation trigger checks and the handoff execution logic.
`src/gemini/client.ts`	Wraps the Google AI SDK. Handles retry, timeout, and JSON parsing.
`src/gemini/prompts.ts`	`buildSystemPrompt(business)` and `buildMessageHistory(conversation)` live here. Modify these when tuning agent behaviour.
`src/gemini/schema.ts`	The strict JSON output schema Gemini must follow. Do not change this without updating `actionRouter.ts`.
`src/whatsapp/webhook.ts`	Inbound message handler. HMAC validation is at the top — don't remove it.
`src/whatsapp/sender.ts`	Outbound message sender. Always use this, never call waapi/WABA directly.
`src/queue/messageWorker.ts`	The BullMQ worker. This is where the async processing pipeline starts.
`src/supabase/queries.ts`	All typed DB queries. Every query here filters by `business_id`. Never add a query that doesn't.
`supabase/migrations/`	SQL migration files. Add a new file for every schema change. Never edit existing migrations.

---

7. WRITING DATABASE QUERIES — RULES

Every query must follow this pattern. No exceptions:

```
// ✅ CORRECT — always scoped to business_id
```

```
async function getConversations(businessId: string) {
  const { data, error } = await supabase
    .from('conversations')
    .select('*')
    .eq('business_id', businessId)   // ← ALWAYS REQUIRED
    .order('last_message_at', { ascending: false });

  if (error) throw new Error(`DB query failed: ${error.message}`);
  return data;
}

// ❌ WRONG — never query without business_id scope
async function getAllConversations() {
  const { data } = await supabase.from('conversations').select('*');
  return data;
}
```

For destructive operations, always go through the confirmation gate:

```
// ❌ WRONG — never execute UPDATE/DELETE directly from agent context
await supabase.from('orders').update({ status: 'cancelled' }).eq('id', orderId);

// ✅ CORRECT — queue a confirmation request first
await queueConfirmationRequest({
  businessId,
  conversationId,
  action: { type: 'update', table: 'orders', id: orderId, data: { status: 'cancel
  message: 'The agent wants to cancel order #1234. Reply CONFIRM or DENY.'
});
```

---

8. WORKING WITH GEMINI

When to modify prompts: Only in `src/gemini/prompts.ts`. Never hardcode prompt text anywhere else.

When to modify the output schema: Only in `src/gemini/schema.ts`. After any change, update `actionRouter.ts` to handle the new fields.

Testing Gemini locally without burning tokens:

```
# Set this in your .env.local to use mock Gemini responses
GEMINI_MOCK=true
```

The mock returns a hardcoded valid JSON response so you can test the full pipeline without API calls.

Debugging bad Gemini outputs: Every Gemini response is logged to `agent_actions` with `action_type: 'gemini_raw'` in development mode. Check Supabase table editor or the audit log in the dashboard to see exactly what Gemini returned on a given turn.

---

9. ADDING A NEW AGENT ACTION

Say you want the agent to be able to look up a customer's order status. Here's the full process:

Step 1 — Add the DB query in `src/supabase/queries.ts`:

```
export async function getOrderById(orderId: string, businessId: string) {
  const { data, error } = await supabase
    .from('orders')
    .select('id, status, items, total, created_at')
    .eq('id', orderId)
    .eq('business_id', businessId)  // always scope
    .single();
  if (error) throw error;
  return data;
}
```

Step 2 — Handle the intent in `src/agent/actionRouter.ts`:

```
case 'query_order':
  const order = await getOrderById(entities.order_id, businessId);
  return formatOrderReply(order);  // turns DB result into natural language
```

Step 3 — Add a reply formatter in `src/agent/formatters.ts`:

```
export function formatOrderReply(order: Order): string {
  // Return a natural-language string, not a JSON dump
  return `Order #${order.id} is currently ${order.status}. It was placed on ${for
}
```

Step 4 — Update the Gemini system prompt in `src/gemini/prompts.ts` to include the new capability in the CAPABILITIES section so the model knows it can query orders.

Step 5 — Write a test in `src/agent/__tests__/actionRouter.test.ts`.

10. RUNNING TESTS

```
cd apps/api

# Run all tests
npm test

# Run tests in watch mode (during development)
npm run test:watch

# Run a specific test file
npm test src/agent/stateMachine.test.ts

# Check test coverage
npm run test:coverage
```

What to test: Every function in `actionRouter.ts`, `escalation.ts`, `stateMachine.ts`, and `queries.ts` must have unit tests. Gemini calls should be mocked in tests using the mock client.

---

11. BRANCHING & PULL REQUEST WORKFLOW

```
main          ← Production. Only merge via reviewed PRs. Never push directly.
staging       ← Pre-production. Deployed to Railway/Vercel staging environments.
dev           ← Integration branch. Merge feature branches here first.
feature/xyz   ← Your working branch. Branch off dev.
fix/xyz       ← Bug fixes. Branch off dev (or main for hotfixes).
```

Before opening a PR:

- All tests pass: `npm test`
- No TypeScript errors: `npm run typecheck`
- Linting passes: `npm run lint`
- You've tested the flow manually using ngrok + a real WhatsApp number
- New DB queries are scoped to `business_id`
- No secrets committed (check with `git diff --staged`)

PR title format: `feat: add order status query action` or `fix: escalation not triggering on keyword match`

---

12. DEPLOYMENT

Backend API → Railway

Deployments to Railway happen automatically via GitHub Actions when a PR is merged to `main`. The workflow is in `.github/workflows/api-deploy.yml`.

To deploy manually:

```
railway up --service blu-bot-api
```

Frontend Dashboard → Vercel

Deployments to Vercel happen automatically on merge to `main`. Vercel picks up the `apps/dashboard` directory.

Environment variables in production

Never put secrets in the codebase. All production env vars are set in:

- Railway dashboard → blu-bot-api → Variables
- Vercel dashboard → blu-bot-dashboard → Environment Variables

Ask the project lead for access to set these.

---

13. DEBUGGING COMMON ISSUES

"Agent not replying to messages"

1. Check BullMQ — is the worker running? (`npm run worker`)
2. Check the job queue in the Redis dashboard — are jobs being enqueued?
3. Check `agent_actions` table — is there a record for the conversation? What's the status?
4. Check Sentry for errors thrown in the worker

"Gemini returning invalid JSON"

1. Check `agent_actions` for `action_type: 'gemini_raw'` — look at the raw response
2. The schema in `src/gemini/schema.ts` may have drifted from what the prompt is asking for
3. Try `GEMINI_MOCK=true` to bypass Gemini and test the rest of the pipeline

"WhatsApp webhook not receiving messages"

1. Is ngrok running and the URL updated in waapi.app?
2. Check the webhook signature validation — is `WHATSAPP_WEBHOOK_SECRET` set correctly?
3. Look at Railway logs for incoming POST requests to `/webhook/whatsapp`

"Dashboard not updating in real time"

1. Check Supabase Realtime is enabled on the `conversations` table (Supabase dashboard → Database → Replication)
2. Check the browser console for Supabase channel subscription errors
3. Confirm the `business_id` filter on the channel matches the logged-in tenant

---

14. CONTACTS & RESOURCES

Resource	Link
Supabase docs	https://supabase.com/docs
Gemini Flash API	https://ai.google.dev/api/generate-content
waapi.app docs	https://waapi.app/docs
Meta WABA docs	https://developers.facebook.com/docs/whatsapp
Hono docs	https://hono.dev/docs
BullMQ docs	https://docs.bullmq.io
Lenco docs	https://lenco.co/developers
Sentry docs	https://docs.sentry.io/platforms/node

Project lead: Contact via the Bluetick Technology Ltd team WhatsApp group for access to accounts, environment variables, and architectural questions.

Engineer guide v1.0 — Bluetick Technology Ltd